

HydIR

A binary lifting workbench for x86-64 ELF

Project overview · September 2026

Project goal

Connect machine bytes to inspectable, testable program artifacts.

- Recover the reachable instructions and control flow of a selected function.
- Lift supported scalar code to LLVM IR and emit readable C.
- Check trusted results against native execution on Linux x86-64.

What HydIR does today

- Imports little-endian x86-64 ELF and reports symbol and segment facts.
- Shows disassembly and a control-flow graph with explicit uncertainty for uncovered bytes.
- Lifts a bounded scalar instruction subset to LLVM IR and scalar C.
- Exposes the work through a CLI, desktop GUI, loopback service, and Python SDK.

How a function moves through HydlR



The recovered CFG keeps branch targets tied to instruction boundaries.

Trusted fixtures add LLVM verification and native execution comparison.

One example: unsigned max

x86-64 assembly

```
hydir_max2:  
movq %rdi, %rax  
cmpq %rsi, %rdi  
jae .Ldone  
movq %rsi, %rax  
.Ldone:  
ret
```

HydIR scalar C

```
uint64_t hydir_lifted(  
uint64_t arg0,  
uint64_t arg1) {  
return arg0 >= arg1  
? arg0 : arg1;  
}
```

Source: tests/fixtures/max2.S and target/demo-video/max2-structured.c

Desktop workbench

- Open a local ELF and browse function facts without uploading the binary.
- Inspect reachable disassembly, CFG, LLVM IR, scalar C, assumptions, and diagnostics.
- Use the Console for status, JSON reports, and a restricted Triton REPL.
- Save pane widths and explicitly reopen the recent local ELF.

Validation on supported scalar code

20

distinct scalar functions

1,008

inputs per function

The local demos compare lifted LLVM and compiled C with native execution on Linux x86-64.

Projects, remote jobs, and edits

- Local projects keep annotations in a private, path-bound SQLite ledger.
- Remote work uses an authenticated loopback service and explicit binary upload.
- Uploads and edits create immutable project revisions; jobs can be monitored or cancelled.
- Named LLVM passes, scalar patching, and rebuilds require separately chosen actions.

Whole-executable recompilation subset

- Three trusted, static freestanding ELF executables exercise the current rebuild path.
- HydlR lifts their decoded .text to stateful LLVM IR and links a bounded runtime.
- The demos compare five controlled executions: stdout, stderr, and exit status.

This is a tested subset, not general ELF recompilation.

Current boundaries

Scalar lift contract

Requires a selected function and an asserted `u64(u64, u64)` prototype.

Supports full-width scalar moves, arithmetic, compare/test, and direct branches.

Rejects unknown semantics or unsafe control-flow recovery.

Not yet covered

High-level structured C recovery.

General ELF recompilation.

Automatic stripped-code discovery.

Whole-program semantic equivalence.

Demo route

- Build the trusted showcase ELF with `scripts/demo-password-showcase.sh`.
- Open it in the GUI, then choose Disassemble ELF.
- Inspect `hydir_policy_route`, `hydir_password_score`, and the Graph view.
- Show `hydir_max2` as a compact assembly-to-C example.

The access-gate phrase and branches are demonstration fixtures, not an authentication system.